TraceForge

How the AI agent observability project works

A practical illustrated guide to running the demo, reading traces, comparing prompt versions, and understanding the regression gate.

Everything runs locally. The sample agent is deterministic and makes no external model calls.

1. What you built

TraceForge is a small observability and evaluation system for an AI-agent workflow. It records a tree of trace spans, redacts sensitive-looking values before storage, evaluates two prompt versions against the same fixed cases, and exposes the results through a local web dashboard and a CI-style gate.

System map

User / CLI calls the agent workflow	Workflow records nested spans	Redactor persists spans to SQLite
Dashboard reads the local API	Evaluation compares prompt versions	Gate checks quality thresholds

The evaluator is deterministic: identical inputs produce identical demo outputs. This makes the project easy to run and inspect without API keys. It demonstrates the observability pattern; it does not claim production model quality.

2. Run it

From PowerShell, open the TraceForge folder, create and activate a virtual environment, then run the demo and dashboard:

```
py -3 -m venv .venv
.\.venv\Scripts\Activate.ps1
.\scripts\run.ps1 demo
.\scripts\run.ps1 serve
```

Open http://127.0.0.1:8000. The demo writes its SQLite database and JSON report under data/generated/. Rerunning the demo replaces only its tagged sample records.

3. Follow one request

A trace is the complete request. It contains spans arranged as parent and child operations. The root workflow span groups retrieval, tool, and model-like work. Each span has a name, timestamps, status, attributes, and optional input/output summaries. The SDK uses context-local state to connect child operations to their parent.

Sensitive-looking keys such as password, token, and API key are redacted before persistence. This is a best-effort safeguard, not a guarantee that arbitrary personal data will be detected.

4. Evaluation and comparison

The bundled JSON cases exercise task success, evidence grounding, and credential safety. The demo evaluates a baseline and a deliberately regressed candidate over the same cases. The report also records illustrative latency and cost values.

The CI gate compares candidate metrics with configured limits. A failed gate is expected for the intentionally regressed sample and returns a nonzero exit code. That gives you a concrete example of blocking a regression before release.

5. Dashboard and endpoints

The standard-library HTTP server binds to loopback by default. The page reads a summary and recent traces from the local API. The demo rerun button creates a fresh report. Endpoints include /api/health, /api/summary, /api/traces, /api/traces/{trace_id}, and POST /api/demo/rerun.

Keep the server on localhost: it has no authentication. Do not expose it to an untrusted network.

6. Folder map

Path	Purpose
src/traceforge/	Agent, SDK, storage, privacy, evaluation, HTTP API, CLI
data/	Fixed synthetic evaluation cases; runtime output is ignored
frontend/	Small local dashboard page
scripts/	PowerShell runner and PDF guide builder
docs/	Architecture, schema, evaluator, CI, privacy, walkthrough
output/pdf/	Generated illustrated guide

7. What to explore next

Inspect docs/architecture.md and docs/trace-schema.md, then run the CLI gate and compare the JSON report. Replace the deterministic agent adapter with a real model provider only after adding provider-specific timeout, retry, cost, and secret-handling policies. Keep real credentials out of the synthetic dataset.

Limitations

This starter is local and single-process, uses SQLite, has no authentication, and has no production load or security validation. The redactor is heuristic. Cost numbers are examples, and the evaluator is not a model judge.